Resenha: Design by Contract (Meyer)

Contratos de software como base para confiabilidade, responsabilidades explícitas e herança segura

Referência

MEYER, Bertrand. **Design by Contract**. In: MANDRIOLI, Dino; MEYER, Bertrand (eds.). *Advances in Object-Oriented Software Engineering*. Prentice Hall, 1991. cap. 1, p. 1-50.

Fonte 1 | Fonte 2. Referências conferidas em 10 out. 2026.

1. Introdução

O capítulo de Bertrand Meyer discute um problema central da engenharia de software: como construir sistemas corretos e robustos sem depender apenas de testes tardios ou de verificações defensivas espalhadas pelo código. A tese principal é que chamadas de rotinas devem ser tratadas como contratos entre cliente e fornecedor, nos quais cada parte possui obrigações e benefícios explícitos. A partir dessa metáfora, o autor organiza uma visão de projeto em que precondições, pós-condições e invariantes tornam visíveis as regras de uso de uma classe, permitindo distribuir responsabilidades, reduzir ambiguidades e aumentar a confiança na evolução de componentes reutilizáveis.

2. Síntese das ideias principais

O texto apresenta primeiro a noção de contrato, depois mostra como assertions expressam esses acordos no código e, em seguida, aplica o mesmo raciocínio ao tratamento de exceções, à herança, ao binding dinâmico e às funções parciais. A contribuição do capítulo está em conectar especificação, implementação e uso de objetos em uma disciplina única de projeto.

2.1 Contratos como divisão explícita de responsabilidades

A metáfora do contrato define que uma rotina não deve ser vista apenas como um bloco de instruções, mas como um serviço com condições claras de acionamento e resultado. O cliente é responsável por chamar a rotina em um estado válido; o fornecedor é responsável por entregar o resultado prometido quando essas condições são respeitadas. Essa divisão protege os dois lados: o cliente sabe o que pode esperar, enquanto o fornecedor não precisa tratar situações fora do escopo contratado. O ganho arquitetural é a redução de regras implícitas, que costumam gerar interpretações diferentes e bugs difíceis de rastrear.

2.2 Assertions, precondições, pós-condições e invariantes

Meyer apresenta as assertions como mecanismo concreto para registrar contratos de software. A precondição define o que precisa ser verdadeiro antes da chamada; a pós-condição define o que será verdadeiro depois da execução; e o invariante de classe expressa propriedades que devem ser preservadas nos estados observáveis dos objetos. Com isso, a especificação deixa de ficar apenas em comentários ou documentação externa e passa a fazer parte da própria interface. Essa ideia é especialmente forte em orientação a objetos, porque a validade de uma classe não depende apenas de cada método isolado, mas da preservação consistente do estado interno ao longo de seu ciclo de vida.

2.3 Crítica à programação defensiva excessiva

Um ponto marcante do capítulo é a crítica à ideia de que toda rotina deve verificar todas as condições possíveis. Para Meyer, se cliente e fornecedor checam a mesma regra, o sistema acumula redundância e complexidade sem necessariamente ganhar confiabilidade. A alternativa é decidir explicitamente quem é responsável por cada condição. Quando a regra pertence à precondição, o cliente deve garanti-la; quando não pertence, o fornecedor deve tratá-la. Essa visão não elimina validações, mas evita verificações duplicadas e transforma a escolha em decisão de arquitetura, guiada pela simplicidade global do sistema.

2.4 Exceções como falhas de contrato, não como fluxo comum

O texto também reorganiza o papel das exceções. Uma exceção não deve ser usada para substituir estruturas normais de controle nem para esconder falhas de uma rotina. Se uma rotina não consegue cumprir seu contrato, ela deve falhar de modo explícito, permitindo que o chamador saiba que o serviço não foi concluído corretamente. A proposta de rescue e retry em Eiffel expressa duas estratégias legítimas: recuperar o estado e tentar novamente, ou restaurar uma condição estável e propagar a falha. O ponto essencial é que o erro não pode ser mascarado como se o contrato tivesse sido cumprido.

2.5 Herança, binding dinâmico e funções parciais

Ao tratar de herança, Meyer interpreta redefinição de métodos como uma forma de subcontratação. Uma subclasse pode oferecer uma implementação melhor, mas não pode exigir mais do cliente nem entregar menos do que a classe original prometia. Por isso, a precondição de uma redefinição deve ser igual ou mais fraca, enquanto a pós-condição deve ser igual ou mais forte. Essa regra preserva a substituibilidade e reduz os riscos do binding dinâmico. No final, o autor também defende funções parciais: em vez de tentar fazer todo componente aceitar qualquer entrada, é mais claro declarar as condições de uso e garantir que as chamadas reais respeitem essas condições.

3. Aplicação imaginada dos conceitos

Em um sistema de agendamento de serviços, a criação de uma reserva poderia ser tratada como um contrato entre o caso de uso e os componentes de domínio. Antes de confirmar o agendamento, o cliente do serviço precisaria garantir condições como data futura, cidade atendida, telefone confirmado, horário disponível e inexistência de reserva pendente conflitante. Em troca, o serviço de domínio garantiria que, após a execução bem-sucedida, a reserva teria identificador, status correto, duração compatível com a cidade e registro persistido em estado consistente. Essas regras poderiam aparecer em testes, validações explícitas, documentação de interfaces e invariantes de entidades.

A aplicação prática mais importante seria evitar validações repetidas e desordenadas em controladores, serviços e repositórios. Algumas verificações pertencem à borda do sistema, como formato de entrada e autenticação; outras pertencem ao domínio, como regras de disponibilidade e transição de status. Separar essas responsabilidades reduz acoplamento e melhora a testabilidade. Em Java, mesmo sem suporte nativo amplo a contratos como Eiffel, a ideia pode ser aplicada com métodos de domínio pequenos, exceções de negócio bem nomeadas, Bean Validation na entrada, testes de contrato e invariantes protegidos por encapsulamento.

4. Conclusão

A principal contribuição de Design by Contract é mostrar que confiabilidade não nasce apenas de tratar erros depois que eles aparecem, mas de tornar explícitas as responsabilidades antes da execução. Ao transformar chamadas de método em acordos verificáveis, Meyer oferece uma base conceitual para projetar classes mais claras, reutilizáveis e seguras diante de evolução. O capítulo é fortemente ligado ao Eiffel, mas sua ideia central continua aplicável em linguagens atuais: bons componentes precisam dizer com precisão o que exigem, o que prometem e quais estados preservam. O cuidado necessário é não transformar contratos em burocracia; seu valor está em simplificar o projeto, não em multiplicar regras formais sem impacto prático.